

Data Structures and Algorithms

Control Structures, Loops, Strings & Random Numbers

1. Control Structures

If-Else Statements

```
// Basic if

if (condition) {

    // code executes if condition is true

}
```

```
// If-else

if (condition) {

    // code if true

} else {

    // code if false

}
```

```
// Else-if chain

if (score >= 90) {

    grade = 'A';

} else if (score >= 80) {

    grade = 'B';

}
```

```
} else if (score >= 70) {  
    grade = 'C';  
} else {  
    grade = 'F';  
}
```

// Nested if

```
if (age >= 18) {  
    if (hasLicense) {  
        cout << "Can drive";  
    }  
}
```

Ternary Operator

```
int max = (a > b) ? a : b; // If a>b true, max=a else max=b  
string status = (age >= 18) ? "Adult" : "Minor";
```

Switch Statement

```
int day = 3;  
switch(day) {  
    case 1:  
        cout << "Monday";  
        break;
```

case 2:

cout << "Tuesday";

break;

case 3:

cout << "Wednesday";

break;

default:

cout << "Invalid day";

break;

}

// Switch with char

char grade = 'A';

switch(grade) {

case 'A':

case 'B':

cout << "Good!";

break;

case 'C':

cout << "Average";

break;

default:

cout << "Fail";

```
}
```

2. Loops

For Loop

```
// Basic for loop
```

```
for (int i = 0; i < 10; i++) {  
    cout << i << " "; // 0 1 2 3 4 5 6 7 8 9  
}
```

```
// Multiple variables
```

```
for (int i = 0, j = 10; i < j; i++, j--) {  
    cout << i << ", " << j; // 0,10 1,9 2,8 3,7 4,6  
}
```

```
// Decrement
```

```
for (int i = 10; i > 0; i--) {  
    cout << i << " "; // 10 9 8 7 6 5 4 3 2 1  
}
```

```
// Nested for loop (2D iteration)
```

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        cout << "(" << i << ", " << j << ") ";  
    }  
}
```

```

    }

    cout << endl;

}

```

While Loop

```
// Basic while

int count = 0;

while (count < 5) {

    cout << count << " ";

    count++; // 0 1 2 3 4

}

// Sentinel-controlled (user input)

int num;

cout << "Enter numbers (0 to stop): ";

cin >> num;

while (num != 0) {

    cout << "You entered: " << num << endl;

    cin >> num;

}

// Infinite loop (use with break)

while (true) {
```

```
// code

if (condition) break;

}
```

Do-While Loop (executes at least once)

```
// Menu example

int choice;

do {

    cout << "1. Option 1\n2. Option 2\n3. Exit\n";

    cout << "Enter choice: ";

    cin >> choice;

    // process choice

} while (choice != 3);
```

```
// Input validation

int num;

do {

    cout << "Enter a positive number: ";

    cin >> num;

} while (num <= 0);
```

Range-based For Loop (C++11)

```
// Arrays
```

```
int arr[] = {1, 2, 3, 4, 5};
```

```
for (int x : arr) {
```

```
    cout << x << " ";
```

```
}
```

```
// Vector
```

```
vector<string> names = {"Alice", "Bob", "Charlie"};
```

```
for (const string& name : names) {
```

```
    cout << name << endl;
```

```
}
```

```
// Auto keyword
```

```
for (auto& num : arr) {
```

```
    num *= 2; // Modify in place
```

```
}
```

Loop Control Statements

```
// Break - exit loop immediately
```

```
for (int i = 0; i < 10; i++) {
```

```
    if (i == 5) break;
```

```
    cout << i << " "; // 0 1 2 3 4
```

```
}
```

```
// Continue - skip current iteration
```

```

for (int i = 0; i < 10; i++) {

    if (i % 2 == 0) continue; // Skip even numbers

    cout << i << " "; // 1 3 5 7 9

}

// Return - exit function (and loop)

void findElement(int arr[], int size, int target) {

    for (int i = 0; i < size; i++) {

        if (arr[i] == target) {

            cout << "Found at " << i;

            return; // Exits function

        }

    }

    cout << "Not found";

}

```

3. Strings in C++

C++ String (std::string)

```

#include <string>

using namespace std;

// Declaration and initialization

string str1;           // Empty string

```



```
string str2 = "Hello";    // Initialize
string str3("World");     // Constructor
string str4(5, 'a');      // "aaaaa"
string str5(str2);        // Copy constructor
```

```
// Basic operations
```

```
string s = "Hello";
cout << s.length();      // 5 (or .size())
cout << s[0];            // 'H'
cout << s.at(1);         // 'e' (bounds checked)
cout << s.back();        // 'o'
cout << s.front();       // 'H'
```

```
// Concatenation
```

```
string s1 = "Hello";
string s2 = "World";
string s3 = s1 + " " + s2; // "Hello World"
s1 += " ";                // "Hello "
s1.append(s2);             // "Hello World"
```

```
// Comparison
```

```
if (s1 == s2) { /* equal */ }
if (s1 != s2) { /* not equal */ }
```

```
if (s1 < s2) { /* lexicographically less */ }  
  
cout << s1.compare(s2);    // 0 if equal, negative if less, positive if greater
```

String Manipulation Functions

```
string s = "Hello World";  
  
// Finding substrings  
  
size_t pos = s.find("World"); // 6 (position)  
  
pos = s.find("x");           // string::npos (not found)  
  
pos = s.find('o');           // 4 (first occurrence)  
  
pos = s.find('o', 5);        // 7 (search from index 5)  
  
pos = s.rfind('o');          // 7 (last occurrence)  
  
  
// Substring  
  
string sub = s.substr(6, 5);  // "World" (start, length)  
  
string sub2 = s.substr(6);    // "World" (to end)  
  
  
// Inserting  
  
s.insert(5, "---");          // "Hello--- World"  
  
s.insert(5, 3, '!');         // "Hello!!! World"  
  
  
// Erasing  
  
s.erase(5, 4);               // Remove 4 chars from position 5
```

```

s.erase(5);           // Remove from position 5 to end

s.clear();             // Empty string

// Replacing

s.replace(6, 5, "C++"); // Replace "World" with "C++"

// Converting case

#include <algorithm>

transform(s.begin(), s.end(), s.begin(), ::toupper); // UPPERCASE

transform(s.begin(), s.end(), s.begin(), ::tolower); // lowercase

// Checking empty

if (s.empty()) { /* string is empty */ }

```

String Input/Output

```

string name;

// cin stops at whitespace

cin >> name;           // "John" from "John Doe"

cout << name;          // "John"

// Get entire line (including spaces)

getline(cin, name);     // "John Doe"

```

```
cout << name;           // "John Doe"
```

```
// With delimiter
```

```
getline(cin, name, ','); // Reads until comma
```

```
// String streams
```

```
#include <sstream>
```

```
string data = "42 3.14 Hello";
```

```
stringstream ss(data);
```

```
int num; double pi; string word;
```

```
ss >> num >> pi >> word; // num=42, pi=3.14, word="Hello"
```

```
// Convert to string
```

```
int num = 42;
```

```
string s = to_string(num); // "42"
```

```
s = to_string(3.14159); // "3.14159"
```

C-String Functions (C-style)

```
#include <cstring>
```

```
char cstr[] = "Hello";
```

```
char dest[20];
```

```
strcpy(dest, cstr);    // Copy

strcat(dest, " World"); // Concatenate

int len = strlen(cstr); // Length (5)

strcmp(cstr, "Hello"); // Compare (0 if equal)

strncpy(dest, cstr, 3); // Copy n chars ("Hel")
```

4. Random Numbers

Modern C++ Random (C++11 - RECOMMENDED)

```
#include <random>

#include <chrono>

// Setup random generator (once in program)

random_device rd;           // Hardware entropy

mt19937 gen(rd());          // Mersenne Twister

// Or seed with time:

mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());

// Integer distribution

uniform_int_distribution<int> dist(1, 100); // [1, 100]

int random_num = dist(gen);

// Real number distribution

uniform_real_distribution<double> realDist(0.0, 1.0);
```

```

double random_double = realDist(gen);    // [0.0, 1.0)

// Normal distribution
normal_distribution<double> normalDist(0.0, 1.0); // Mean=0, Std=1
double normal = normalDist(gen);

// Bernoulli distribution
bernoulli_distribution bernoulli(0.5);    // 50% chance
bool coin_flip = bernoulli(gen);

// Example: Dice roll
uniform_int_distribution<int> die(1, 6);
int roll = die(gen);

// Example: Random character
uniform_int_distribution<int> charDist(65, 90); // A-Z
char randomChar = static_cast<char>(charDist(gen));

```

Old C-style Random (NOT RECOMMENDED)

```

#include <cstdlib>

#include <ctime>

// Seed once at program start

```

```

srand(time(nullptr)); // Seed with current time

// Generate random number

int rand_num = rand();           // [0, RAND_MAX]

int dice = rand() % 6 + 1;       // [1, 6] (biased!)

int range = rand() % 100;        // [0, 99] (biased!)

// Better (less biased)

int better = rand() / (RAND_MAX / 100 + 1); // [0, 100]

```

Random Number Examples

```

#include <random>

#include <chrono>

#include <vector>

#include <algorithm>

class RandomGenerator {
private:
    mt19937 gen;
public:
    RandomGenerator() {
        auto seed = chrono::steady_clock::now().time_since_epoch().count();
        gen = mt19937(seed);
    }
}

```

```
int getInt(int min, int max) {  
    uniform_int_distribution<int> dist(min, max);  
    return dist(gen);  
}
```

```
double getDouble(double min, double max) {  
    uniform_real_distribution<double> dist(min, max);  
    return dist(gen);  
}
```

```
bool getBool(double probability = 0.5) {  
    bernoulli_distribution dist(probability);  
    return dist(gen);  
}  
};
```

// Usage

```
RandomGenerator randGen;  
  
int randomNum = randGen.getInt(1, 100);  
  
double randomDouble = randGen.getDouble(0.0, 1.0);  
  
bool heads = randGen.getBool(0.5);
```



```

// Randomize container

vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

shuffle(vec.begin(), vec.end(), randGen.gen); // Random shuffle


// Generate random string

string generateRandomString(int length) {

    const string chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

                        "abcdefghijklmnopqrstuvwxyz"

                        "0123456789";

    random_device rd;

    mt19937 gen(rd());

    uniform_int_distribution<int> dist(0, chars.length() - 1);


    string result;

    for (int i = 0; i < length; i++) {

        result += chars[dist(gen)];

    }

    return result;

}

```

5. Combined Examples

Guessing Game

```

#include <iostream>

#include <random>

```

```
#include <string>

using namespace std;

int main() {

    random_device rd;

    mt19937 gen(rd());

    uniform_int_distribution<int> dist(1, 100);

    int secret = dist(gen);

    int guess;

    int attempts = 0;

    string input;

    cout << "Guess the number (1-100): ";

    do {

        getline(cin, input);

        guess = stoi(input);

        attempts++;

        if (guess < secret) {

            cout << "Too low! Try again: ";

        } else if (guess > secret) {
```

```

        cout << "Too high! Try again: ";

    } else {

        cout << "Correct! You took " << attempts << " attempts!";

    }

} while (guess != secret);

return 0;

}

```

Name Generator

```

#include <iostream>

#include <vector>

#include <random>

#include <string>

using namespace std;

int main() {

    vector<string> firstNames = {"Alice", "Bob", "Charlie", "Diana", "Eve"};

    vector<string> lastNames = {"Smith", "Johnson", "Williams", "Brown",
"Jones"};

    random_device rd;

    mt19937 gen(rd());

    uniform_int_distribution<int> firstDist(0, firstNames.size() - 1);

```

```

uniform_int_distribution<int> lastDist(0, lastNames.size() - 1);

// Generate 10 random names
for (int i = 0; i < 10; i++) {

    string fullName = firstNames[firstDist(gen)] + " " +

        lastNames[lastDist(gen)];

    cout << fullName << endl;

}

return 0;

}

```

Password Validator

```

string validatePassword() {

    string password;

    while (true) {

        cout << "Enter password: ";

        getline(cin, password);

        if (password.length() < 8) {

            cout << "Too short! Need at least 8 characters.\n";

            continue;

        }

    }
}

```

```
bool hasUpper = false, hasLower = false, hasDigit = false;

for (char c : password) {

    if (isupper(c)) hasUpper = true;

    if (islower(c)) hasLower = true;

    if (isdigit(c)) hasDigit = true;

}


if (!hasUpper) {

    cout << "Need an uppercase letter.\n";

    continue;

}

if (!hasLower) {

    cout << "Need a lowercase letter.\n";

    continue;

}

if (!hasDigit) {

    cout << "Need a digit.\n";

    continue;

}


break; // Valid password

}

return password;
```

}

Best Practices Summary

Concept	Best Practice
If-Else	Use early returns to avoid deep nesting
Switch	Always include default and break
Loops	Prefer range-based for loops for containers
Strings	Use std::string over C-strings in C++
Random	Use <random> (C++11+), NOT rand()
Input	Use getline() for string input with spaces

Quick Reference: Common Functions

// Strings

s.length() // size

s.empty() // check empty

s.find(sub) // find substring

s.substr(pos, len) // extract substring

s.append(str) // append

s.insert(pos, str) // insert at position

s.erase(pos, len) // remove characters

s.replace(pos, len, str) // replace

// Random (Modern)

```
uniform_int_distribution<int> dist(min, max);
```

```
uniform_real_distribution<double> dist(min, max);
```

```
bernoulli_distribution dist(0.5);
```

```
normal_distribution<double> dist(mean, std_dev);
```

Functions

Pointers in C/C++

What is a Pointer?

A pointer is a variable that stores the memory address of another variable (or function), rather than storing a value directly.

```
int x = 42;    // Regular variable storing value 42
int* ptr = &x; // Pointer storing the address of x
```

Pointer Syntax Basics

Operator	Name	Purpose
*	Asterisk/Dereference	Declares a pointer OR gets value at address
&	Address-of	Gets memory address of a variable
->	Arrow	Dereference + member access (for structs/classes)

```
int value = 10;
int* ptr = &value; // & gets address
*ptr = 20;         // * dereferences (value now 20)
```

Types of Pointers

1. Null Pointer

```
int* ptr = nullptr; // C++11 (preferred)
```

```
int* ptr = NULL;    // C-style  
  
int* ptr = 0;       // Old style  
  
// Always initialize! Dereferencing null = crash
```

2. Void Pointer (void*)

Generic pointer that can point to any data type (must cast to use):

```
int x = 5;  
  
void* generic = &x;  
  
int* specific = (int*)generic; // Cast required  
  
cout << *specific;           // 5
```

3. Pointer to Pointer (Double Pointer)

```
int x = 10;  
  
int* ptr = &x;  
  
int** pptr = &ptr; // Points to the pointer  
  
cout << **pptr;    // 10
```

4. Function Pointer

```
int add(int a, int b) { return a + b; }  
  
int (*funcPtr)(int, int) = &add;  
  
cout << funcPtr(3, 4); // 7 (calls add)
```

5. Smart Pointers (C++)

```
#include <memory>

std::unique_ptr<int> uptr = std::make_unique<int>(10);

std::shared_ptr<int> sptr = std::make_shared<int>(20);

std::weak_ptr<int> wptr = sptr; // Non-owning
```

Pointer Arithmetic

Only valid for arrays (same type):

```
int arr[] = {10, 20, 30, 40};

int* ptr = arr;        // Points to arr[0]

cout << *ptr;          // 10

ptr++;                 // Moves to next int (4 bytes typically)

cout << *ptr;           // 20

cout << *(ptr + 1);     // 30 (ptr+1 is next element)


// Pointer difference gives element count:

int* start = arr;

int* end = arr + 4;

cout << end - start;    // 4 (number of elements)
```

Pointers vs Arrays

```
int arr[5] = {1,2,3,4,5};

int* ptr = arr;
```

// Key differences:

cout << sizeof(arr); // 20 (5 * 4 bytes)

cout << sizeof(ptr); // 8 (pointer size on 64-bit)

arr++; // ERROR! Array name is constant

ptr++; // OK! Pointer can move

Pointers and const

const int x = 10;

int y = 20;

// Pointer to constant data (can't change value)

const int* p1 = &x;

*p1 = 15; // ERROR! Can't modify const data

p1 = &y; // OK! Can point elsewhere

// Constant pointer (can't change address)

int* const p2 = &y;

*p2 = 30; // OK! Can modify value

p2 = &x; // ERROR! Can't change address

// Constant pointer to constant data

const int* const p3 = &x;

```
*p3 = 5;    // ERROR! Can't change value
```

```
p3 = &y;    // ERROR! Can't change address
```

Dynamic Memory Allocation

C-style (malloc/free)

```
int* p = (int*)malloc(5 * sizeof(int)); // Allocate
if (p != NULL) {
    // Use p
    free(p); // Deallocate
}
```

C++-style (new/delete)

```
int* p = new int(10);    // Single int initialized to 10
```

```
int* arr = new int[5];    // Array of 5 ints
```

```
delete p;                // Delete single
```

```
delete[] arr;            // Delete array
```

Memory Leak Example

```
void leak_example() {
    int* p = new int(100);

    // Forgot to delete! Memory leak!

} // p goes out of scope but memory isn't freed
```

Common Pointer Pitfalls

1. Dangling Pointer (pointing to freed memory)

```
int* ptr = new int(5);
```

```
delete ptr;
```

```
*ptr = 10; // UNDEFINED BEHAVIOR! (use-after-free)
```

2. Uninitialized Pointer

```
int* ptr; // Contains garbage address
```

```
*ptr = 10; // CRASH! (wild pointer)
```

3. Buffer Overflow

```
int arr[5];
```

```
for(int i=0; i<10; i++) {
```

```
    arr[i] = i; // Overwrites memory beyond array!
```

```
}
```

4. Returning Address of Local Variables

```
int* bad_function() {
```

```
    int x = 5;
```

```
    return &x; // WRONG! x destroyed when function returns
```

```
}
```

Pointers in Functions

Pass by Pointer (modify original)

```
void swap(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int x=5, y=10;
```

```
swap(&x, &y); // x=10, y=5
```

Pointer to Pointer (modify pointer itself)

```
void allocate(int** p) {  
    *p = new int(42);  
}
```

```
int* ptr = nullptr;
```

```
allocate(&ptr); // ptr now points to new int
```

```
cout << *ptr; // 42
```

```
delete ptr;
```

Smart Pointers vs Raw Pointers

Feature	Raw Pointer	unique_ptr	shared_ptr
Memory Management	Manual	Automatic (RAII)	Reference counted
Copy	Shallow	Move-only	Shallow (shared)
Ownership	Unclear	Exclusive	Shared
Null Safety	None	Can check	Can check
Overhead	None	Minimal	Atomic ref counting
Use Case	Observing, C APIs	Exclusive ownership	Shared ownership

```
// Modern C++ recommendation:
std::unique_ptr<int> uptr = std::make_unique<int>(10);
// Use raw pointers only for non-owning observations
int* raw = uptr.get(); // Observe, DON'T delete
```

Pointer to Member (Class)

```
class MyClass {
public:
    int value = 5;

    void print() { cout << value; }
};
```

// Pointer to member variable

```
int MyClass::*ptrMember = &MyClass::value;

MyClass obj;

cout << obj.*ptrMember;    // 5
```

// Pointer to member function

```
void (MyClass::*ptrFunc)() = &MyClass::print;

(obj.*ptrFunc)();          // 5
```

Best Practices

1. Always initialize pointers (nullptr if no target)
2. Check for null before dereferencing
3. Prefer references when you don't need pointer arithmetic

4. Use smart pointers in modern C++ (avoid raw new/delete)
5. Be careful with pointer arithmetic (stay within array bounds)
6. Free allocated memory exactly once
7. Don't mix malloc/free with new/delete
8. Use const to document and enforce intent

Quick Reference Card

// Declaration

int* p; // Pointer to int

const int* p; // Pointer to const int

int* const p; // Const pointer to int

const int* const p; // Const pointer to const int

// Common Operations

&var // Address of var

*ptr // Value at ptr

ptr->mem // Member access (equivalent to (*ptr).mem)

ptr[n] // Array indexing (equivalent to *(ptr+n))

// Casts

(int*)voidPtr // C-style cast

static_cast<int*>(voidPtr) // C++ cast

reinterpret_cast<int*>(ptr) // Low-level cast

String and C Strings

C-String (C-style String)

- Definition: Null-terminated character array (char* or char[])
- Termination: Ends with '\0' (null character)
- Memory: Manual management required
- Length: Determined by traversing until '\0'
- Header: <cstring> or <string.h>

// C-String examples

```
char str1[] = "Hello";    // Array of 6 chars (includes '\0')
```

```
char* str2 = (char*)malloc(20);
```

```
strcpy(str2, "World");
```

```
// Must manually free: free(str2);
```

C++ String (std::string)

- Definition: Class object from STL
- Termination: Stores size internally, no null terminator needed
- Memory: Automatic management (RAII)
- Length: length() or size() method (O(1))
- Header: <string>

// C++ String examples

```
std::string s1 = "Hello";    // Object with dynamic memory
```

```
std::string s2 = "World";
```

```
std::string s3 = s1 + " " + s2; // Concatenation
```

// Automatically cleaned up

Key Differences Table

Feature	C-String	std::string
Type	Character array	Class object
Memory	Manual (malloc/free)	Automatic (RAII)
Length	O(n) via strlen()	O(1) via .size()
Safety	Buffer overflow risk	Bounds checked
Operations	C functions (strcpy, strcat)	Methods (.append(), .find())
Concatenation	strcat() or manual	+ operator
Comparison	strcmp()	==, !=, <, etc.
Modification	Limited, manual	Rich methods
Copy	strcpy()	Assignment, copy constructor
Performance	Faster for simple ops	Slightly slower, but safer

When to Use Which

Use C-Strings when:

- Working with C APIs or embedded systems
- Performance-critical low-level operations
- Memory-constrained environments
- Interfacing with OS/system calls

Use std::string when:

- Writing modern C++ code
- Need dynamic manipulation (append, insert, replace)
- Want safety and convenience
- Using STL algorithms and containers

Conversion Between Types

C-String → std::string

```
const char* cstr = "Hello";  
  
std::string str(cstr);    // Constructor  
  
std::string str2 = cstr;  // Assignment
```

std::string → C-String

```
std::string str = "Hello";  
  
const char* cstr = str.c_str(); // Read-only, null-terminated  
  
char* mutable_cstr = str.data(); // C++17, mutable
```

Common Pitfalls

C-String Issues:

```
char buffer[10];  
  
strcpy(buffer, "This is too long!"); // BUFFER OVERFLOW!
```

std::string Issues:

```
std::string s;  
  
s[10] = 'x'; // Undefined behavior (access out of bounds)
```

// Use .at() for bounds checking: s.at(10) = 'x'; // throws exception

Performance Comparison

Operation	C-String	std::string
Creation	Fast	Slower (constructor overhead)
Length	O(n)	O(1)
Copy	Fast (memcpy)	May allocate memory
Concatenation	O(n+m)	O(n+m) with possible reallocation

Best Practice in Modern C++

Prefer std::string for safety, readability, and maintainability. Use C-strings only when necessary for C compatibility or when extreme performance is required.